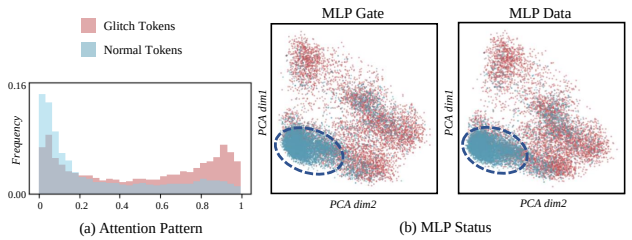


Figure 1: The distribution of attention patterns and MLP status for glitch tokens (shown in red color) and normal tokens (in blue color) in Llama2 model

(2) Submitting the prompt to the model, which is configured by setting the temperature parameter to zero.
(3) Observing and analyzing the model's output to determine whether it accurately reproduces the input token. The output is deemed successful if the model returns the exact token as requested; otherwise, the token is classified as a glitch token.

We traverse all the 32,000 tokens of the Llama2 model and eventually identify 6,425 glitch tokens from the entire vocabulary.To precisely capture intermediate layer outputs within the model, we resort to a transformer mechanistic interpretability tool named *Transformer-lens* [30]. Its hook technique enables real-time access of the activation values at all layers and allows code insertion into specific intermediate layers of the model. In this study, we insert hooks into all intermediate layers during the first forward of the tested model. This approach is chosen because the first forward comprehensively reflects the model's understanding of the input sequence and highlights the differences between normal and glitch tokens.

We select two key features to represent the model's internal output, i.e., *attention patterns* and *MLP status*. The attention patterns capture the relative importance and relationships between tokens, while MLP status is composed of MLP gate and MLP data (Section 2.1), providing insights on how the model synthesizes and modulates new representations within the MLP module.

3.2 RQ1: Glitch Token Characteristics

We compare the extracted intermediate results of Llama2 model when processing prompts containing normal tokens and glitch tokens and observe significant disparity in attention patterns and MLP status. The distribution of attention patterns for glitch tokens in some attention heads differs significantly from that of normal tokens. To quantitatively analyze these differences, we randomly sample normal tokens in size of the same number as the pre-identified glitch tokens. To analyze the attention patterns, we create two sets of prompts: one containing 6,425 prompts with glitch tokens and another containing 6,425 prompts with normal tokens. We then compute the frequency distribution of attention patterns generated by these prompts, categorizing them into different value ranges. We visualize the results using a histogram, as shown in Figure 1 (a). The attention patterns of normal tokens (shown in blue color) generally cluster around lower ranges and exhibit a relatively smooth distribution. In contrast, the attention patterns for glitch tokens (in red color) display a comparably divergent and chaotic distribution.

Furthermore, the distribution characteristics of glitch tokens in the MLP status show deviations compared to normal tokens. Due to the high dimension of MLP status values, we cannot visualize them in the same method used for attention patterns. Instead, we resort to PCA algorithm to convert the captured MLP status, i.e., MLP gate and MLP data, to two dimension values. We present the distribution of MLP gate and MLP data in scatter plots, as shown in Figure 1 (b). It can be observed that both two representations of MLP status for normal tokens tend to cluster towards a centroid, forming a relatively dense and bounded distribution. In contrast, the MLP status of glitch tokens are highly dispersed and scattered.

Finding 1

Llama2 shows a significant disparity in attention patterns and MLP status when dealing with glitch tokens and normal tokens.

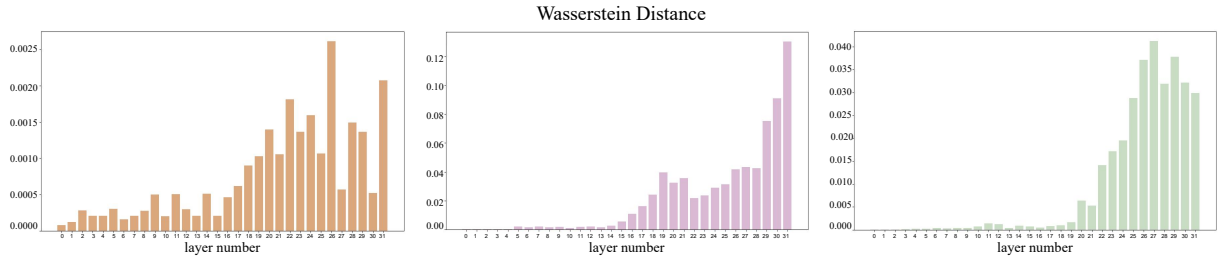
To illustrate the anomalies across layers, we resort to the Wasserstein distance [38] to measure the magnitude of the differences in the intermediate layers outputs produced by normal and glitch tokens, and thereby reveal the distinctions in the model's internal mechanisms when processing these two groups of tokens. In this study, a larger Wasserstein distance indicates a greater distributional difference. Figure 2 shows the Wasserstein distance in attention patterns and MLP status between normal and glitch tokens across different layers of the Llama2 model. We find that the differences caused by normal and glitch tokens per layer are not uniformly distributed. The attention patterns and MLP status exhibit greater differences in the downstream layers closer to the output, e.g., layers 19-31. This finding suggests that the impact of glitch tokens, although may result in negligible erroneous results in front layers, is amplified along with the propagation, leading to unexpected outputs in the end.

Finding 2

The anomalous intermediate results caused by glitch tokens are not uniformly distributed across all layers of the model but are concentrated and amplified in specific key layers.

3.3 RQ2: Ubiquity

In order to verify whether our previous findings exist in other LLMs, two additional LLMs, namely Qwen-7B-Chat model and Mistral-7B-Instruct model (shortly as Qwen and Mistral), are selected to complement our empirical study. As shown in Figure 3, the experimental results on these two models are similar to those of Llama2. The attention patterns of glitch tokens and normal tokens exhibit inconsistent distribution. For example, the attention patterns of normal tokens mainly fall within the range of $[0, 0.2]$ in the Qwen model, while the attention patterns of glitch tokens show a different shape and concentrates in the range of $[0.8, 1]$. Such inconsistency can also be observed in the Mistral model, evidenced by the attention values of normal tokens and glitch tokens primarily approximating around 0.5 and 0.7, respectively. In terms of MLP status, the intermediate layers tend to produce outputs in a centroid-based cluster for normal tokens. However, the MLP status values of glitch tokens are overall chaotic and disseminated.



(a) Attention Pattern (b) MLP Gate (c) MLP Data

Figure 2: Wasserstein distance of the probability distributions between glitch tokens and normal tokens in different intermediate layers of Llama2 model

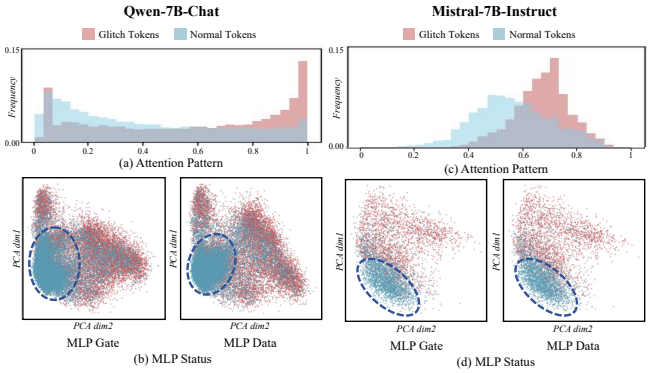


Figure 3: The example distribution of attention patterns, MLP gate and MLP data for glitch tokens (shown in red color) and normal tokens (in blue color) in Qwen-7B-Chat and Mistral-7B-Instruct.

> **Finding 3**
>
> We identify similar differences exhibited between normal and glitch tokens at the intermediate layers of different LLMs.

The findings of RQ1 provide insights for the subsequent detection and fix of glitch tokens, while the finding in RQ2 offers factual evidence for the broad application of our approach in LLMs.

4 METHODOLOGY

Based on the findings from our empirical study, we propose the GlitchProber algorithm, which aims to achieve automatic detection and fix of glitch tokens by analyzing the internal activation states of LLMS. Our approach consists of two main ideas:

(1) Leveraging the differences in model activation values when processing glitch tokens and normal tokens to achieve rapid screening of glitch tokens. By designing an anomaly detection algorithm, we can identify and label potential glitch tokens based on their activation features extracted from specific layers, which we refer to as key layers (detailed in Section 4.3). These key layer features are crucial for detecting glitch tokens.
(2) Fixing errors caused by glitch tokens by adjusting the model's intermediate results. We designed a series of experiments where we automatically adjusted the output values of the model's intermediate layers and observed the impact on the final output.

As elucidated in Section 3.2, glitch tokens in the model predominantly affect the downstream layers close to the output, notably impacting attention patterns and MLP status in specific key layers. This revelation is instrumental in shaping our approach of GlitchProber. By strategically focusing our efforts on key layers, we can significantly reduce computational overhead while achieving a comparable level of detection and fix efficacy to traversing all layers. Thus, this approach not only optimizes the efficiency of our methods but also ensures that our interventions are targeted where they are most needed. For a detailed introduction to the selection strategy of key layers, please refer to Section 4.3.

Based on these ideas, we designed GlitchProber, which integrates detection and fix algorithms. GlitchProber's detection algorithm identifies and locates glitch tokens causing model output errors by analyzing intermediate layer activation states. It randomly samples a subset of tokens from the vocabulary, tests them using repetitive tasks, and extracts activation features from key layers. These features undergo dimensionality reduction and are labeled based on repetitive task outcomes. A classifier is then trained with the labeled data to assess unknown tokens. Finally, the remaining tokens are detected individually using the trained SVM classifier, and predictions are verified through repetitive tasks.

GlitchProber's fix algorithm corrects the anomalous activation patterns of glitch tokens by adjusting activation values in the model's intermediate layers, eliminating their negative impact on the model output. It first calculates activation statistics of normal tokens in key layers and identifies neurons that are consistently activated or silent in most normal tokens. Then, it compares these neurons' activations between normal and glitch tokens, calculating suppression ratio coefficients for anomalous activations and activation values to promote silent neurons. Finally, it rectifies the activation values of glitch tokens in key layers based on these coefficients and values, automatically fixing the glitch tokens.

4.1 Detecting Glitch Tokens via GlitchProber

GlitchProber identifies and detects glitch tokens that cause model output errors by analyzing the intermediate layer activation states of Transformer language models when processing tokens. The main workflow of the algorithm is shown in Figure 4. The detection algorithm of GlitchProber consists of three main steps: feature extraction and dimension reduction, SVM-based glitch token classifier, and glitch token identification and validation.

4.1.1 ***Feature Extraction and Dimension Reduction.*** GlitchProber adopts a random sampling strategy to select samples from

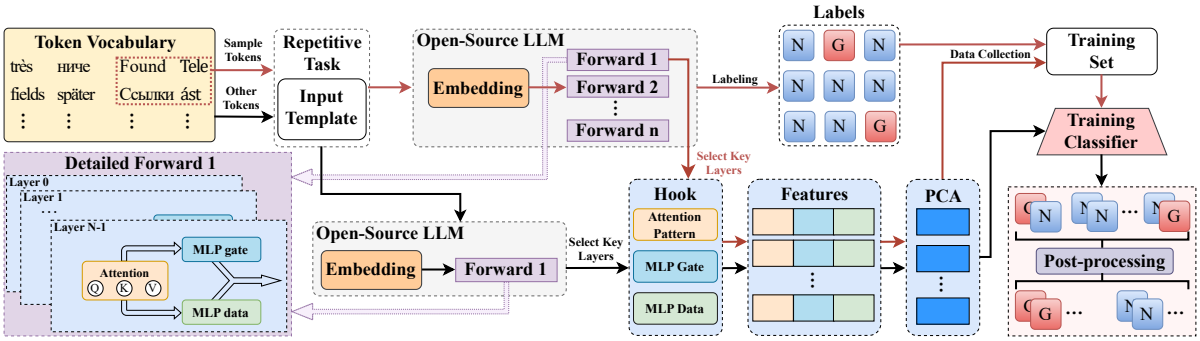


Figure 4: GlitchProber workflow for detecting glitch tokens. The red arrows represent the data flow during the training process, while the black arrows represent the data flow during the detection process.

the model's token vocabulary V to form the sample set S. The choice of sampling rate γ needs to balance between sample size and computational efficiency. A larger γ leads to a larger sample size and more accurate detection results but also incurs higher computational costs. Conversely, a smaller γ results in a smaller sample size and faster computation but may affect the detection performance. Through experiments, we determined that when γ is in the range of [0.1, 0.3], GlitchProber achieves a good balance between detection performance and efficiency.

GlitchProber uses a unified repetitive task to construct input sequences for glitch token identification in the sample set S. For tokens in the sample set S, the algorithm assigns corresponding category labels according to the output results of the repetitive task. At the same time, we extract the attention pattern, MLP gate and MLP data features in the model's first forward process. However, the dimension of the original feature tensors is high, and directly using them to train the classifier would lead to excessive computational costs. To improve computational efficiency, we adopt a dimension reduction strategy. The PCA algorithm [1] is applied to reduce the dimension, mapping the original high-dimensional features to a low-dimensional subspace while maximally preserving the discriminative information of the features. Through experiments, we found that when the dimension P of the reduced features is in the range of [50, 200], a good balance between computational efficiency and information retention can be achieved. Therefore, we set $P = 75$ as the default dimension reduction parameter.

4.1.2 ***SVM-based Glitch Token Classifier.*** GlitchProber trains an SVM classifier using the low-dimensional feature representation matrix F of the sample set S and the corresponding category labels. The trained SVM classifier will be used for subsequent glitch assessment of unknown tokens. SVM is a binary classification algorithm that is particularly suitable for problems with high-dimensional feature spaces, which aligns well with the characteristics of the glitch token detection task in GlitchProber. Besides, SVM has higher computational efficiency and shorter training time when handling high-dimensional features compared to other binary classification algorithms. That helps achieve rapid real-time detection of glitch tokens in GlitchProber. Therefore, we adopt SVM as the classifier in the detection algorithm of GlitchProber.

4.1.3 ***Glitch Token Identification and Validation.*** The tokens in the token vocabulary that were not sampled are individually detected. The token to be detected is input into the same repetitive task as in the training phase, and its features attention patterns, MLP gate, and MLP data are extracted in the model's first forward module. Subsequently, the trained SVM classifier is used to make predictions based on the extracted features.

If a token is predicted as "glitchy", the algorithm will input this token into the model and further use the repetitive task to validate it. If the model can correctly repeat the token, we consider it as a potential normal token, and the SVM classifier may have made a false positive prediction. Through this post-processing step, GlitchProber can effectively reduce the false positive rate of glitch token detection and improve the precision of detection. Finally, GlitchProber outputs two sets: the set of glitch tokens G and the set of normal tokens N.

Algorithm 1: GlitchProber (Detection)

```
Input: Token Vocabulary V; PCA Dimension P; Sample Rate γ;
       KeyLayers[]
Output: Glitch token set G; Normal token set N
S ← randomSample(V, γ);
foreach Token ∈ S do
    SampleFeatures ← hookModel(Token, KeyLayers);
    SampleLabels ← validateGlitch(Token);
end
F ← PCA(SampleFeatures, P);
Classifier ← trainClassifier(F, SampleLabels);
foreach Token ∈ V do
    if Token ∉ S then
        Feature ← PCA((hookModel(Token, KeyLayers)), P);
        if classify(Classifier, Feature) == 'Normal' then
            N ← Token;
        else
            if validateGlitch(Token) == 'Glitch' then
                G ← Token;
            else
                N ← Token;
            end
        end
    end
end
```

4.1.4 ***Detection Process Algorithm.*** The pseudocode for the detection algorithm of GlitchProber is shown in Algorithm 1.